

Universidade do Minho
Escola de Engenharia

Cálculo de Programas

Trabalho Prático (2025/26)

Lic. em Ciências da Computação
Lic. em Engenharia Informática

Grupo G99

a106807	Marco Sèvegrand
axxxxxx	Nome
axxxxxx	Nome

Preâmbulo

Em [Cálculo de Programas](#) pretende-se ensinar a programação de computadores como uma disciplina científica. Para isso parte-se de um repertório de *combinadores* que formam uma álgebra da programação e usam-se esses combinadores para construir programas *composicionalmente*, isto é, agregando programas já existentes.

Na sequência pedagógica dos planos de estudo dos cursos que têm esta disciplina, opta-se pela aplicação deste método à programação em [Haskell](#) (sem prejuízo da sua aplicação a outras linguagens funcionais). Assim, o presente trabalho prático coloca os alunos perante problemas concretos que deverão ser implementados em [Haskell](#). Há ainda um outro objectivo: o de ensinar a documentar programas, a validá-los e a produzir textos técnico-científicos de qualidade.

Antes de abordarem os problemas propostos no trabalho, os grupos devem ler com atenção o anexo [A](#) onde encontrarão as instruções relativas ao *software* a instalar, etc.

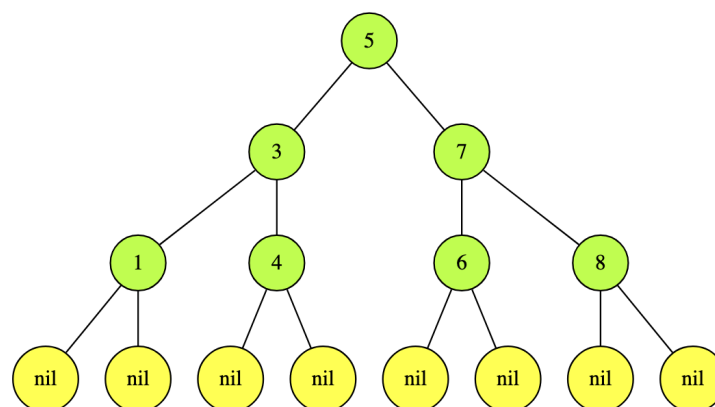
Valoriza-se a escrita de *pouco* código que corresponda a soluções simples e elegantes que utilizem os combinadores de ordem superior estudados na disciplina.

Avaliação. Faz parte da avaliação do trabalho a sua defesa por parte dos elementos de cada grupo. Estes devem estar preparados para responder a perguntas sobre *qualquer* dos problemas deste enunciado. A prestação *individual* de cada aluno nessa defesa oral será uma componente importante e diferenciadora da avaliação.

Problema 1

Uma serialização (ou travessia) de uma árvore é uma sua representação sob a forma de uma lista. Na biblioteca *BTree* encontram-se as funções de serialização *inordt*, *preordt* e *postordt*, que fazem as travessias *in-order*, *pre-order* e *post-order*, respectivamente. Todas essas travessias são catamorfismos que percorrem a árvore argumento em regime *depth-first*.

Pretende-se agora uma função *bfordr* que faça a travessia em regime *breadth-first*, isto é, por níveis. Por exemplo, para a árvore t_1 dada em anexo e mostrada na figura a seguir,



a função deverá dar a lista

[5, 3, 7, 1, 4, 6, 8]

em que se vê como os níveis 5, depois 3, 7 e finalmente 1, 4, 6, 8 foram percorridos.

Pretendemos propor duas versões dessa função:

1. Uma delas envolve um catamorfismo de *BTrees*:

$$\begin{aligned} \text{bfsLevels} &:: \text{BTree } a \rightarrow [a] \\ \text{bfsLevels} &= \text{concat} \cdot \text{levels} \end{aligned}$$

Complete a definição desse catamorfismo:

$$\begin{aligned} \text{levels} &:: \text{BTree } a \rightarrow [[a]] \\ \text{levels} &= \llbracket g\text{levels} \rrbracket \end{aligned}$$

2. A segunda proposta,

$$\text{bft} :: \text{BTree } a \rightarrow [a]$$

deverá basear-se num anamorfismo de listas.

Sugestão: estudar o artigo [?] cujo PDF está incluído no material deste trabalho. Quando fizer testes ao seu código pode, se desejar, usar funções disponíveis na biblioteca *Exp* para visualizar as árvores em GraphViz (formato .dot).

Justifique devidamente a sua resolução, que deverá vir acompanhada de diagramas explicativos. Como já se disse, valoriza-se a escrita de *pouco* código que corresponda a soluções simples e elegantes que utilizem os combinadores de ordem superior estudados na disciplina.

Problema 2

Considere a seguinte função em Haskell:

```
f x = wrapper · worker where
  wrapper = head
  worker 0 = start x
  worker (n + 1) = loop x (worker n)
  loop x [s, h, k, j, m] =
    [h / k + s, x ↑ 2 * h, k * j, j + m, m + 8]
  start x = [x, x ↑ 3, 6, 20, 22]
```

Pode-se provar pela lei de recursividade mútua que $f\ x\ n$ calcula o seno hiperbólico de x , $\sinh x$, para n aproximações da sua série de Taylor. Faça a derivação da função dada a partir da referida série de Taylor, apresentando todos os cálculos justificativos, tal como se faz para outras funções no capítulo respectivo do texto base desta UC [?].

Problema 3

Quem em Braga observar, ao fim da tarde, o tráfego onde a Avenida Clairmont Fernand se junta à N101, aproximadamente na coordenada [41°33'46.8"N 8°24'32.4"W](#) — ver as setas da figura que se segue — reparará nas sequências imparáveis (infinitas!) de veículos provenientes dessas vias de circulação.

Mas também irá observar um comportamento interessante por parte dos condutores desses veículos: por regra, *cada carro numa via deixa passar, à sua frente, exactamente outro carro da outra via*.



Este comportamento *civilizado* chama-se *fair-merge* (ou *fair-interleaving*) de duas sequências infinitas, também designadas *streams* em ciência da computação. Seja dado o tipo dessas sequências em Haskell,

data *Stream* *a* = *Cons* (*a*, *Stream* *a*) **deriving** *Show*

para o qual se define também:

out (*Cons* (*x*, *xs*)) = (*x*, *xs*)

O referido comportamento civilizado pode definir-se, em Haskell, da forma seguinte:¹

```
fair_merge :: (Stream a, Stream a) + (Stream a, Stream a) → Stream a
fair_merge = [h, k] where
  h (Cons (x, xs), y) = Cons (x, k (xs, y))
  k (x, Cons (y, ys)) = Cons (y, h (x, ys))
```

Defina *fair_merge* como um **anamorfismo** de *Streams*, usando o combinador

$\llbracket g \rrbracket = \text{Cons} \cdot (\text{id} \times \llbracket g \rrbracket) \cdot g$

e a seguinte estratégia:

- Derivar a lei **dual** da recursividade mútua,

$$[f, g] = \llbracket [h, k] \rrbracket \equiv \begin{cases} \text{out} \cdot f = F [f, g] \cdot h \\ \text{out} \cdot g = F [f, g] \cdot k \end{cases} \quad (1)$$

tal como se fez, nas aulas, para a que está no formulário.

- Usar (1) na resolução do problema proposto.

Justificar devidamente a resolução, que deverá vir acompanhada de diagramas explicativos.

Problema 4

Como se sabe, é possível pensarmos em catamorfismos, anamorfismos etc *probabilísticos*, quer dizer, programas recursivos que dão distribuições como resultados. Por exemplo, podemos pensar num combinador

pcataList :: (() + (*a*, *b*) → *Dist* *b*) → [*a*] → *Dist* *b*

¹ O facto das sequências serem infinitas não nos deve preocupar, pois em Haskell isso é lidado de forma transparente por [lazy evaluation](#).

que é muito parecido com

$$(\cdot) :: () \rightarrow (a, b) \rightarrow b \rightarrow [a] \rightarrow b$$

da biblioteca [List](#). A principal diferença é que o gene de *pcataList* é uma função probabilística.

Como exemplo de utilização, recorde-se que ([zero, add]) soma todos os elementos da lista argumento, por exemplo:

$$(\text{[zero, add]}) [20, 10, 5] = 35.$$

Considere-se agora a função *padd* (adição probabilística) que, com probabilidade 90% soma dois números e com probabilidade 10% os subtrai:

$$\text{padd } (a, b) = D [(a + b, 0.9), (a - b, 0.1)]$$

Se se correr

$$d4 = \text{pcataList } [\text{pzero, padd}] [20, 10, 5] \text{ where } \text{pzero} = \text{return} \cdot \text{zero}$$

obter-se-á:

```
35  81.0%
25   9.0%
 5   9.0%
15   1.0%
```

Com base neste exemplo, resolva o seguinte

Problema: Uma unidade militar pretende enviar uma mensagem urgente a outra, mas tem o aparelho de telegrafia meio avariado. Por experiência, o telegrafista sabe que a probabilidade de uma palavra se perder (não ser transmitida) é 5%; e que, no final de cada mensagem, o aparelho envia o código "stop", mas (por estar meio avariado), falha 10% das vezes.

Qual a probabilidade de a palavra "atacar" da mensagem

`words "Vamos atacar hoje"`

se perder, isto é, o resultado da transmissão ser ["Vamos", "hoje", "stop"]? E a de seguirem todas as palavras, mas faltar o "stop" no fim? E a da transmissão ser perfeita?

Responda a estas perguntas encontrando *gene* tal que

`transmitir = pcataList gene`

descreve o comportamento do aparelho. Justificar devidamente a resolução, que deverá vir acompanhada de diagramas explicativos.

Anexos

A Natureza do trabalho a realizar

Este trabalho teórico-prático deve ser realizado por grupos de 3 alunos. Os detalhes da avaliação (datas para submissão do relatório e sua defesa oral) são os que forem publicados na [página da disciplina](#) na internet.

Recomenda-se uma abordagem participativa dos membros do grupo em **todos** os exercícios do trabalho, para assim poderem responder a qualquer questão colocada na *defesa oral* do relatório.

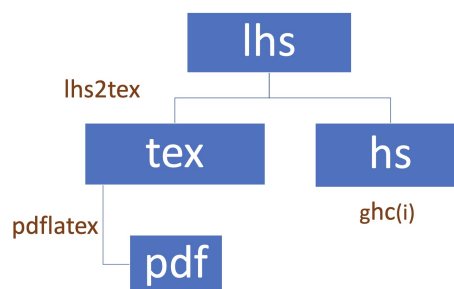
Para cumprir de forma integrada os objectivos do trabalho vamos recorrer a uma técnica de programação dita “**literária**” [?], cujo princípio base é o seguinte:

Um programa e a sua documentação devem coincidir.

Por outras palavras, o **código fonte** e a **documentação** de um programa deverão estar no mesmo ficheiro.

O ficheiro `cp2526t.pdf` que está a ler é já um exemplo de **programação literária**: foi gerado a partir do texto fonte `cp2526t.lhs`¹ que encontrará no **material pedagógico** desta disciplina descompactando o ficheiro `cp2526t.zip`.

Como se mostra no esquema abaixo, de um único ficheiro (*lhs*) gera-se um PDF ou faz-se a interpretação do código **Haskell** que ele inclui:



Vê-se assim que, para além do **GHCI**, serão necessários os executáveis **pdflatex** e **lhs2TeX**. Para facilitar a instalação e evitar problemas de versões e conflitos com sistemas operativos, é recomendado o uso do **Docker** tal como a seguir se descreve.

B Docker

Recomenda-se o uso do **container** cuja imagem é gerada pelo **Docker** a partir do ficheiro `Dockerfile` que se encontra na diretoria que resulta de descompactar `cp2526t.zip`. Este **container** deverá ser usado na execução do **GHCI** e dos comandos relativos ao **L^AT_EX**. (Ver também a `Makefile` que é disponibilizada.)

Após **instalar o Docker** e descarregar o referido zip com o código fonte do trabalho, basta executar os seguintes comandos:

```
$ docker build -t cp2526t .  
$ docker run -v ${PWD}:/cp2526t -it cp2526t
```

NB: O objetivo é que o container seja usado *apenas* para executar o **GHCI** e os comandos relativos ao **L^AT_EX**. Deste modo, é criado um *volume* (cf. a opção `-v ${PWD}:/cp2526t`) que permite que a diretoria em que se encontra na sua máquina local e a diretoria `/cp2526t` no **container** sejam partilhadas.

Pretende-se então que visualize/edite os ficheiros na sua máquina local e que os compile no **container**, executando:

¹ O sufixo ‘lhs’ quer dizer *literate Haskell*.

```
$ lhs2TeX cp2526t.lhs > cp2526t.tex
$ pdflatex cp2526t
```

[lhs2TeX](#) é o pre-processor que faz “pretty printing” de código Haskell em [L^AT_EX](#) e que faz parte já do [container](#). Alternativamente, basta executar

```
$ make
```

para obter o mesmo efeito que acima.

Por outro lado, o mesmo ficheiro `cp2526t.lhs` é executável e contém o “kit” básico, escrito em [Haskell](#), para realizar o trabalho. Basta executar

```
$ ghci cp2526t.lhs
```

Abra o ficheiro `cp2526t.lhs` no seu editor de texto preferido e verifique que assim é: todo o texto que se encontra dentro do ambiente

```
\begin{code}
...
\end{code}
```

é seleccionado pelo [GHCi](#) para ser executado.

C Em que consiste o TP

Em que consiste, então, o *relatório* a que se referiu acima? É a edição do texto que está a ser lido, preenchendo o anexo [G](#) com as respostas. O relatório deverá conter ainda a identificação dos membros do grupo de trabalho, no local respectivo da folha de rosto.

Para gerar o PDF integral do relatório deve-se ainda correr os comando seguintes, que actualizam a bibliografia (com [BibT_EX](#)) e o índice remissivo (com [makeindex](#)),

```
$ bibtex cp2526t.aux
$ makeindex cp2526t.idx
```

e recompilar o texto como acima se indicou. (Como já se disse, pode fazê-lo correndo simplesmente `make` no [container](#).)

No anexo [F](#) disponibiliza-se algum código [Haskell](#) relativo aos problemas que são colocados. Esse anexo deverá ser consultado e analisado à medida que isso for necessário.

Deve ser feito uso da [programação literária](#) para documentar bem o código que se desenvolver, em particular fazendo diagramas explicativos do que foi feito e tal como se explica no anexo [D](#) que se segue.

D Como exprimir cálculos e diagramas em L^AT_EX/lhs2TeX

Como primeiro exemplo, estudar o texto fonte ([lhs](#)) do que está a ler¹ onde se obtém o efeito seguinte:²

$$\begin{aligned} id &= \langle f, g \rangle \\ \equiv \quad &\{ \text{universal property} \} \end{aligned}$$

¹ Procure e.g. por “`sec:diagramas`”.

² Exemplos tirados de [?].

$$\begin{aligned}
& \begin{cases} \pi_1 \cdot id = f \\ \pi_2 \cdot id = g \end{cases} \\
\equiv & \quad \{ \text{identity} \} \\
& \begin{cases} \pi_1 = f \\ \pi_2 = g \end{cases} \\
& \square
\end{aligned}$$

Os diagramas podem ser produzidos recorrendo à *package* [xymatrix](#), por exemplo:

$$\begin{array}{ccc}
\mathbb{N}_0 & \xleftarrow{\text{in}} & 1 + \mathbb{N}_0 \\
\downarrow \langle g \rangle & & \downarrow id + \langle g \rangle \\
B & \xleftarrow{g} & 1 + B
\end{array}$$

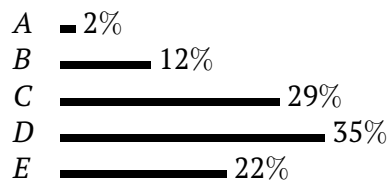
E O mónade das distribuições probabilísticas

Mónades são funtores com propriedades adicionais que nos permitem obter efeitos especiais em programação. Por exemplo, a biblioteca [Probability](#) oferece um mónade para abordar problemas de probabilidades. Nesta biblioteca, o conceito de distribuição estatística é captado pelo tipo

$$\text{newtype Dist } a = D \{ \text{unD} :: [(a, \text{ProbRep})] \} \quad (2)$$

em que *ProbRep* é um real de 0 a 1, equivalente a uma escala de 0 a 100%.

Cada par (a, p) numa distribuição $d :: \text{Dist } a$ indica que a probabilidade de a é p , devendo ser garantida a propriedade de que todas as probabilidades de d somam 100%. Por exemplo, a seguinte distribuição de classificações por escalões de A a E ,



será representada pela distribuição

$$\begin{aligned}
d1 &:: \text{Dist Char} \\
d1 &= D [('A', 0.02), ('B', 0.12), ('C', 0.29), ('D', 0.35), ('E', 0.22)]
\end{aligned}$$

que o [GHCi](#) mostrará assim:

```

'D'  35.0%
'C'  29.0%
'E'  22.0%
'B'  12.0%
'A'   2.0%

```

É possível definir geradores de distribuições, por exemplo distribuições *uniformes*,

$$d2 = \text{uniform} (\text{words "Uma frase de cinco palavras"})$$

isto é


```

    "Uma"    20.0%
    "cinco"  20.0%
    "de"     20.0%
    "frase"  20.0%
    "palavras" 20.0%

```

distribuição *normais*, eg.

```
d3 = normal [10..20]
```

etc.¹ Dist forma um **mónade** cuja unidade é $\text{return } a = D [(a, 1)]$ e cuja composição de Kleisli é (simplificando a notação)

$$(f \bullet g) a = [(y, q * p) \mid (x, p) \leftarrow g a, (y, q) \leftarrow f x]$$

em que $g : A \rightarrow \text{Dist } B$ e $f : B \rightarrow \text{Dist } C$ são funções **monádicas** que representam *computações probabilísticas*.

Este mónade é adequado à resolução de problemas de *probabilidades e estatística* usando programação funcional, de forma elegante e como caso particular da programação monádica.

F Código fornecido

Problema 1

Árvores exemplo:

```

t1 :: BTree Int
t1 = Node (5, (Node (3, (Node (1, (Empty, Empty)), Node (4, (Empty, Empty)))),
    Node (7, (Node (6, (Empty, Empty)), Node (8, (Empty, Empty)))))
t2 :: BTree Int
t2 =
    node 1
        (node 2 (node 4 Empty Empty) (node 5 Empty Empty))
        (node 3 (node 6 Empty Empty) (node 7 Empty Empty))
t3 :: BTree Char
t3 =
    node 'A'
        (node 'B' (node 'C' (node 'D' Empty Empty) Empty) Empty)
        (node 'E' Empty Empty)
t4 :: BTree Char
t4 =
    node 'A'
        (node 'B' (node 'C' (node 'D' Empty Empty) Empty) Empty)
        Empty
t5 :: BTree Int
t5 =
    node 1

```

¹ Para mais detalhes ver o código fonte de [Probability](#), que é uma adaptação da biblioteca [PFP](#) ("Probabilistic Functional Programming"). Para quem quiser saber mais recomenda-se a leitura do artigo [?].

```

(node 2 (node 4 Empty Empty) Empty)
(node 3 Empty (node 5 (node 6 Empty Empty) Empty))
node a b c = Node (a, (b, c))

```

G Soluções dos alunos

Os alunos devem colocar neste anexo as suas soluções para os exercícios propostos, de acordo com o “layout” que se fornece. Não podem ser alterados os nomes ou tipos das funções dadas, mas pode ser adicionado texto ao anexo, bem como diagramas e/ou outras funções auxiliares que sejam necessárias.

Importante: Não pode ser alterado o texto deste ficheiro fora deste anexo.

Problema 1

Uma travessia em largura (breadth-first) de uma árvore processa os nós nível por nível, começando pela raiz. Existem diferentes formas de implementar esta travessia, cada uma explorando um combinador recursivo distinto.

Primeira abordagem: catamorfismo de árvores

A função *levels* agrupa elementos por profundidade, retornando uma lista de listas onde cada sub-lista representa um nível. Depois, *bfsLevels* concatena simplesmente estes níveis numa única lista, obtendo o resultado breadth-first.

Embora o resultado final seja breadth-first, o catamorfismo processa internamente a árvore de baixo para cima (visitando primeiro as subárvores). Quando chegamos a um nó, temos já processadas as subárvores esquerda e direita, recebendo listas de níveis para cada uma. O nó atual forma um novo nível no topo, e os restantes níveis são fundidos par a par através de *zipLevels*, alinhando elementos da mesma profundidade das duas subárvores.

Implementação:

```

glevels :: () + (a, ([[a]], [[a]])) -> [[a]]
glevels (i1 ()) = []
glevels (i2 (a, (ls, rs))) = [a] : zipLevels ls rs
where
  zipLevels [] rs = rs
  zipLevels ls [] = ls
  zipLevels (l : ls) (r : rs) = (l ++ r) : zipLevels ls rs

```

Diagrama e Propriedades:

Catamorfismo *levels*:

$$\begin{array}{ccc}
 BTree\ a & \xleftarrow{outBTree} & 1 + a \times (BTree\ a \times BTree\ a) \\
 \downarrow levels & & \downarrow id + id \times (levels \times levels) \\
 [[a]] & \xleftarrow{glevels} & 1 + a \times ([[a]] \times [[a]])
 \end{array}$$

O diagrama mostra como *levels* decompõe a árvore (via *outBTree*), aplica *levels* recursivamente às subárvores, e depois combina o resultado usando *glevels*. O gene *glevels* trata dois casos: árvore vazia

(retorna lista vazia) e nó com valor a e subárvores processadas (cria um novo nível com a e funde os restantes).

Segunda abordagem: anamorfismo de listas

A função *bft* implementa uma travessia breadth-first através de um anamorfismo que utiliza uma fila de árvores. Ao contrário do catamorfismo que destrói a estrutura, o anamorfismo constrói incrementalmente a lista resultado.

O algoritmo funciona da seguinte forma: iniciamos com uma fila contendo apenas a raiz da árvore. Depois, repetidamente, removemos a árvore da frente da fila. Se for *Empty*, ignoramo-la e prosseguimos. Se for um nó *Node*, extraímos o seu valor (que é emitido para a lista resultado) e adicionamos os seus filhos no fim da fila. Este regime FIFO (first-in-first-out) garante que visitamos os nós exactamente na ordem breadth-first: primeiro todos os nós do nível 0 (raiz), depois nível 1, depois nível 2, e assim sucessivamente.

Implementação:

```

bft t = [(gbf)] [t]
where
  gbf [] = i1 ()
  gbf (Empty : ts) = gbf ts
  gbf (Node (a, (l, r)) : ts) = i2 (a, ts ++ [l, r])

```

Diagrama e Propriedades:

Anamorfismo *bft*:

$$\begin{array}{ccc}
 [BTree\ a] & \xleftarrow{gbf} & 1 + a \times [BTree\ a] \\
 \downarrow [(gbf)] & & \downarrow id + id \times [(gbf)] \\
 [a] & \xleftarrow{inList} & 1 + a \times [a]
 \end{array}$$

O gene *gbf* define o passo de construção da lista. A fila de árvores é consumida por três casos: se está vazia, o processo termina (i_1 ()); se contém uma árvore vazia, ignoramo-la recursivamente; se contém um nó, emitimos o seu valor e continuamos com os restantes elementos da fila mais os seus filhos. A operação $ts ++ [l, r]$ é essencial: coloca os filhos no fim, não no início, mantendo a ordem breadth-first.

Problema 2

O objetivo deste problema é derivar a implementação da função seno hiperbólico ($\sinh x$) a partir da sua série de Taylor, utilizando a lei da recursividade mútua e catamorfismos de números naturais.

A série de Taylor para $\sinh x$

A série de Taylor do seno hiperbólico é:

$$\sinh x = \sum_{n=0}^{\infty} \frac{x^{2n+1}}{(2n+1)!} = x + \frac{x^3}{3!} + \frac{x^5}{5!} + \dots$$

Para calcular esta série de forma iterativa e eficiente, mantemos um estado que evolui em cada iteração, e usamos um catamorfismo de números naturais para repetir este processo n vezes.

Estrutura do estado iterativo

O estado é representado por um vector de cinco componentes $[s, h, k, j, m]$:

- s : Soma acumulada dos termos da série.
- h : Numerador do termo actual (x^{2n+1}).
- k : Denominador do termo actual ($((2n + 1)!)^2$).
- j : Produto dos dois próximos factores da sequência de denominadores.
- m : Incremento linear para j .

A transição de um estado para o seguinte é dada pela função *loop*:

$$\text{novo estado} = \left[\frac{h}{k} + s, x^2 h, k j, j + m, m + 8 \right]$$

Estado inicial para $n = 0$:

$$\text{start } x = [x, x^3, 6, 20, 22]$$

Este estado inicial contém já o primeiro termo da série ($s = x$) e prepara o cálculo do segundo termo (com $h = x \uparrow 3$ e $k = 6 = 3!$).

Implementação:

```
f_sinh :: Double → Int → Double
f_sinh x n = head (worker n)
  where
    worker = ([start x, loop x])
    loop x state = case state of
      [s, h, k, j, m] → [h / k + s, x ↑ 2 * h, k * j, j + m, m + 8]
    start x = [x, x ↑ 3, 6, 20, 22]
```

Diagrama do catamorfismo:

$$\begin{array}{ccc} \mathbb{N}_0 & \xleftarrow{\text{in}} & 1 + \mathbb{N}_0 \\ \text{worker} \downarrow & & \downarrow \text{id} + \text{worker} \\ [\text{Double}] & \xleftarrow{\text{gene}} & 1 + [\text{Double}] \end{array}$$

O catamorfismo *worker* aplica iterativamente a função *loop x* começando no estado inicial *start x*. A função *gene* trata dois casos: para o caso base (zero), retorna o estado inicial; para o caso indutivo, aplica *loop x* ao estado resultante da iteração anterior. Por fim, *head* extrai o primeiro componente do vector final, que é a soma acumulada s .

Justificação matemática

A série de $\sinh x$ converge rapidamente. A relação entre termos consecutivos é:

$$\frac{t_{n+1}}{t_n} = \frac{x^2}{(2n + 2)(2n + 3)}$$

Esta razão decrescente garante que, após um número finito de iterações, os novos termos contribuem negligenciavelmente para a soma, permitindo uma aproximação numérica precisa de $\sinh x$.

Problema 3

O problema do fair-merge de duas sequências infinitas (Streams) descreve uma situação de alternância perfeita entre dois fluxos de dados, como observado no tráfego onde cada carro de uma via deixa passar um carro da outra via. O objetivo é definir esta operação como um anamorfismo de Streams, utilizando a lei dual da recursividade mútua.

A lei dual da recursividade mútua

A propriedade universal de um anamorfismo estabelece que uma função $[f, g]$ pode ser expressa como um anamorfismo quando o seu comportamento externo segue uma certa estrutura. Para streams, a lei dual da recursividade mútua afirma:

$$\begin{aligned} [f, g] &= \llbracket [h, k] \rrbracket \\ \equiv & \{ \text{propriedade universal} \} \\ \text{out} \cdot [f, g] &= (id \times [f, g]) \cdot [h, k] \\ \equiv & \{ \text{fusão de somas} \} \\ & \begin{cases} \text{out} \cdot f = (id \times [f, g]) \cdot h \\ \text{out} \cdot g = (id \times [f, g]) \cdot k \end{cases} \\ & \square \end{aligned}$$

Esta lei permite-nos derivar o gene do anamorfismo a partir de duas funções mutuamente recursivas.

Aplicação ao fair-merge

O fair-merge alterna entre dois fluxos. As funções h e k que descrevem este comportamento são:

- $h (Cons (x, xs), y) = Cons (x, k (xs, y))$: extrai x do primeiro fluxo e passa para k .
- $k (x, Cons (y, ys)) = Cons (y, h (x, ys))$: extrai y do segundo fluxo e passa para h .

Aplicando out (que satisfaz $out (Cons (a, as)) = (a, as)$):

- $out \cdot h (Cons (x, xs), y) = (x, k (xs, y))$
- $out \cdot k (x, Cons (y, ys)) = (y, h (x, ys))$

Para construir o gene, utilizamos o isomorfismo $\cdot + \cdot$ para alternar entre os dois estados:

- Quando o gene recebe i_1 , extrai de h e retorna o resultado encapsulado em i_2 .
- Quando o gene recebe i_2 , extrai de k e retorna o resultado encapsulado em i_1 .

O gene implementa assim a alternância definida por h e k :

- $gene (i_1 (Cons (x, xs), y)) = (x, i_2 (xs, y))$
- $gene (i_2 (x, Cons (y, ys))) = (y, i_1 (x, ys))$

Implementação:

$$\begin{aligned} fair_merge' &= \llbracket gene \rrbracket \\ \text{where} \\ gene (i_1 (Cons (x, xs), y)) &= (x, i_2 (xs, y)) \\ gene (i_2 (x, Cons (y, ys))) &= (y, i_1 (x, ys)) \end{aligned}$$

Diagrama do anamorfismo:

$$\begin{array}{ccc}
 (Stream\ a, Stream\ a) + (Stream\ a, Stream\ a) & \xrightarrow{gene} & a \times (Stream\ a, Stream\ a) + (Stream\ a, Stream\ a) \\
 \downarrow fair_merge' & & \downarrow id \times fair_merge' \\
 Stream\ a & \xleftarrow{Cons} & a \times Stream\ a
 \end{array}$$

O diagrama ilustra o passo do anamorfismo: o estado inicial é uma soma (escolha entre dois modos). O gene *gene* processa este estado, extrai um elemento e produz um novo estado alternado. O anamorfismo constrói iterativamente a stream final, preservando a alternância garantida pelo gene.

Funcionamento

Cada aplicação de *gene* extrai um elemento de um dos fluxos e inverte a escolha (via $\cdot + \cdot$) para o próximo passo, garantindo assim que os elementos são consumidos alternadamente dos dois fluxos de entrada. Esta é a essência do fair-merge civilizado observado no problema do tráfego.

Problema 4

Este problema envolve catamorfismos probabilísticos aplicados a um cenário de transmissão de mensagens com falhas.

Análise do Problema

Uma unidade militar pretende enviar a mensagem *words* "Vamos atacar hoje" através de um aparelho de telegrafia com falhas probabilísticas:

- Cada palavra tem 5% de probabilidade de se perder (95% de sucesso)
- O código "stop" final tem 10% de probabilidade de falha (90% de sucesso)

Catamorfismo Probabilístico

O catamorfismo *pcataList* processa listas de forma recursiva com comportamento probabilístico. A assinatura de tipo e implementação já foram definidas anteriormente:

```

pcataList g = h
where
  h [] = g (i1 ())
  h (x : xs) = h xs >>= λr → g (i2 (x, return r))

```

Este combinador é semelhante a *foldr* da biblioteca *List*, mas o gene é uma função probabilística que retorna distribuições.

Implementação do Gene

O gene descreve como processar cada elemento da lista recursivamente:

```

gene :: () + (String, [String]) → Dist [String]
gene (i1 ()) = choose 0.9 ["stop"] []
gene (i2 (w, tail)) = choose 0.95 (w : tail) tail

```

Explicação Detalhada do Gene

O gene tem dois casos correspondentes à estrutura de soma $\cdot + \cdot$:

1. **Caso base** *gene* (i_1 ()):

Quando não há mais palavras, o aparelho tenta enviar "stop":

- Com probabilidade 90%: envia ["stop"] com sucesso
- Com probabilidade 10%: falha e retorna [] (nada enviado)

2. **Caso recursivo** *gene* (i_2 (*word*, *distTail*)):

Recebe uma palavra e uma distribuição de probabilidade da cauda já processada. A composição monádica $\text{distTail} \gg \lambda \text{tail} \rightarrow \dots$ garante que:

- Para cada possível resultado *tail* da cauda
- Com probabilidade 95%: a palavra é transmitida, retorna *word* : *tail*
- Com probabilidade 5%: a palavra se perde, retorna apenas *tail*

Análise Probabilística para ["Vamos", "atacar", "hoje"]

Cada resultado final é formado pelas decisões independentes de transmissão de cada palavra e do "stop". As probabilidades multiplicam-se conforme a semântica do mónade Dist.

1. **Transmissão perfeita:** ["Vamos", "atacar", "hoje", "stop"]

Todas as 3 palavras são transmitidas com sucesso E o "stop" é transmitido:

$$P = 0.95 \times 0.95 \times 0.95 \times 0.90 = 0.7712 = 77.12\%$$

Processamento pela recursão de *pcataList*:

- $h []: \text{gene } (i_1 ()) \rightarrow \text{choose } 0.9 \text{ ["stop"]} []$
- $h ["hoje"]: \text{choose } 0.95 \text{ ["hoje", "stop"]} ["stop"] (0.95)$
- $h ["atacar", "hoje"]: \text{choose } 0.95 \text{ ["atacar", "hoje", "stop"]} ["hoje", "stop"] (0.95 \times 0.95)$
- $h ["Vamos", "atacar", "hoje"]: \text{resultado final } (0.95 \times 0.95 \times 0.95 \times 0.90)$

2. **Falha no "stop":** ["Vamos", "atacar", "hoje"]

Todas as 3 palavras são transmitidas MAS o "stop" falha (10%):

$$P = 0.95 \times 0.95 \times 0.95 \times 0.10 = 0.0857 = 8.57\%$$

Processamento:

- $h []: \text{gene } (i_1 ()) \rightarrow \text{choose } 0.9 \text{ ["stop"]} [] \text{ escolhe } [] \text{ (probabilidade 0.1)}$
- $h ["hoje"]: \text{obtem } [], \text{ transmite "hoje"} \rightarrow ["hoje"]$
- $h ["atacar", "hoje"]: \text{obtem } ["hoje"], \text{ transmite "atacar"} \rightarrow ["atacar", "hoje"]$
- $h ["Vamos", "atacar", "hoje"]: \text{obtem } ["atacar", "hoje"], \text{ transmite "Vamos"} \rightarrow ["Vamos", "atacar", "hoje"]$

3. Perda de "atacar": ["Vamos", "hoje", "stop"]

Apenas a palavra "atacar" se perde (5%), as outras transmitem-se (95% cada):

$$P = 0.95 \times 0.05 \times 0.95 \times 0.90 = 0.0406 = 4.06\%$$

Processamento:

- $h []: gene (i_1 ()) \rightarrow choose\ 0.9\ ["stop"] []$ escolhe ["stop"] (probabilidade 0.9)
- $h ["hoje"]:$ obtém ["stop"], transmite "hoje" $\rightarrow ["hoje", "stop"]$
- $h ["atacar", "hoje"]:$ obtém ["hoje", "stop"], perde "atacar" $\rightarrow ["hoje", "stop"]$
- $h ["Vamos", "atacar", "hoje"]:$ obtém ["hoje", "stop"], transmite "Vamos" $\rightarrow ["Vamos", "hoje", "stop"]$

Testes em Haskell

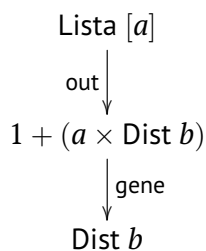
```
transmissao = transmitir (words "Vamos atacar hoje")
probPerfecta = (≡ ["Vamos", "atacar", "hoje", "stop"]) ?? transmissao
probSemStop = (≡ ["Vamos", "atacar", "hoje"]) ?? transmissao
probPerdaAtacar = (≡ ["Vamos", "hoje", "stop"]) ?? transmissao
```

Resultados esperados:

- $probPerfecta \approx 0.7712$ (77.12%)
- $probSemStop \approx 0.0857$ (8.57%)
- $probPerdaAtacar \approx 0.0406$ (4.06%)

Diagrama Explicativo

O catamorfismo *pcataList* segue o padrão clássico de fold, destruindo a estrutura de lista recursivamente e acumulando resultados probabilísticos:



A recursão processa a lista de trás para a frente:

$$pcataList\ g\ [a_1, a_2, a_3] = g(\text{Right}(a_1, g(\text{Right}(a_2, g(\text{Right}(a_3, g(\text{Left}()))))))))$$

Justificação da Solução

A função *gene* encapsula correctamente o comportamento probabilístico do aparelho:

1. **Uso de $\cdot + \cdot$:** A soma $() + (a, \text{Dist } b)$ representa:
 - $i_1 ()$: caso base, sem mais elementos
 - $i_2 (\text{word}, \text{distTail})$: elemento actual e distribuição da cauda
 2. **Composição monádica:** A expressão $\text{distTail} \gg \lambda \text{tail} \rightarrow \dots$ garante que:
 - Cada resultado possível de distTail é processado
 - As probabilidades são multiplicadas automaticamente pelo mónade
 - A semântica é intuitiva e composicional
 3. **Independência probabilística:** As falhas de cada palavra e do "stop" são modeladas como eventos independentes. A função *choose* multiplica as probabilidades correctamente.
 4. **Catamorfismo genérico:** *pcataList* é um catamorfismo que funciona com qualquer gene $g :: () + (a, \text{Dist } b) \rightarrow \text{Dist } b$. O gene *gene* especializa-o para transmissão de mensagens.
 5. **Laziness em Haskell:** A definição recursiva funciona graças à avaliação preguiçosa (lazy evaluation) de Haskell. As distribuições infinitas são tratadas de forma transparente.
 6. **Definição de transmitir:** A função $\text{transmitir} = \text{pcataList } \text{gene}$ (definida na linha 383) compõe o catamorfismo genérico com o gene específico para este problema, obtendo a solução desejada.
- A solução mostra como padrões funcionais clássicos (catamorfismos) podem ser elevados a contextos probabilísticos, mantendo a elegância e composicionalidade do código.